Informatique Théorique

CM4: Représentation des entiers

Florian Bridoux

Polytech Nice Sophia

2025-2026

Table des matières

1 Changement de base

2 Représentation finie

Table des matières

1 Changement de base

2 Représentation finie

Les bases

Quand on veut écrire des nombres, il faut d'abord choisir dans quelle base b on veut les représenter. L'entier b correspond au nombre de chiffres ou symboles distincts disponibles dans cette base.

Par exemple :

- La base 10 (que vous utilisez tous les jours) utilise les chiffres $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$.
- La base 2 (binaire) les symbole 0 et 1.
- La base 16 (hexadécimale) utilise les symboles $0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F$.
- La base 1 (unaire) (que vous utilisez quand vous comptez sur vos doigts), le symbole 1.

Les bases

Quand on écrit 8394 en base 10, on représente l'entier :

$$8 * 1000 + 3 * 100 + 9 * 10 + 4 * 1$$

Autrement dit,

$$8 * 10^3 + 3 * 10^2 + 9 * 10^1 + 4 * 10^0.$$

D'une manière générale dans une base b, un entier s'écrit

$$d_m d_{m-1} d_{m-2} \dots d_1 d_0$$

où d_i est un symbole représentant un entier entre 0 et $b-1$. Sa valeur est alors :

$$\sum_{0 \leq i \leq m} d_i * b^i.$$

Conversion d'une base vers la base 10

Pour passer d'une base b à une base 10, il suffit de calculer :

$$\sum_{0 \leq i \leq m} d_i * b^i.$$

Exemples :

- $(11001)_2 = (1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 0 * 2^1 + 1 * 2^0)_{10} = (2^4 + 2^3 + 2^0)_{10} = (16 + 8 + 1)_{10} = (25)_{10}$.
- $(A6E)_{16} = (10 * 16^2 + 6 * 16^1 + 14 * 16^0)_{10} = (10 * 256 + 6 * 16 + 14)_{10} = (2560 + 96 + 14)_{10} = (2670)_{10}$.

Conversion de la base 10 vers une autre base

Comment convertir un nombre n en base 10 en base b ?
Méthode 1 : division successive de n par b :

1. On prend $i = 0$.
2. On fait la division euclidienne de n par b, ça nous donne un reste r et un quotient q.
3. $d_i \leftarrow r$, $n \leftarrow q$, $i \leftarrow i + 1$.
4. Si $n = 0$ on s'arrête. Sinon on reprend à l'étape (2).
5. On obtient $d_m d_{m-1} \dots d_1$.

Conversion de la base 10 vers une autre base

Exemple : $(28)_{10} = (?)_2$

Division	Quotient	Reste
28 / 2	14	0
14 / 2	7	0
7 / 2	3	1
3 / 2	1	1
1 / 2	0	1

Résultat : $(28)_{10} = (11100)_2$

Remarque 1 : il ne faut pas oublier de lire les reste de bas en haut et pas de haut en bas.

Remarque 2 : avec cette méthode on calcule d'abord les bits de poids faibles.

Conversion de la base 10 vers une autre base

Méthode 2 : On cherche la plus grande puissance de b plus petite que n et on la soustrait.

- Trouver k tel que $b^k \leq n < b^{k+1}$.
- Calculer le coefficient $c_k = \left\lfloor \frac{n}{b^k} \right\rfloor$.
- Soustraire $c_k \cdot b^k$ de n.
- Répéter le processus pour le reste jusqu'à obtenir 0.

Exemple : $(45)_{10}$ en base 3.

- $3^3 = 27 \quad \Rightarrow \quad c_3 = 1, \quad 45 - 1 \cdot 27 = 18.$
- $3^2 = 9 \quad \Rightarrow \quad c_2 = 2, \quad 18 - 2 \cdot 9 = 0.$

Résultat : $(45)_{10} = (1200)_3$.

Conversion de la base 10 vers une autre base

Méthode 2 : On cherche la plus grande puissance de b plus petite que n et on la soustrait.

- Trouver k tel que $b^k \leq n < b^{k+1}$.
- Calculer le coefficient $c_k = \left\lfloor \frac{n}{b^k} \right\rfloor$.
- Soustraire $c_k \cdot b^k$ de n.
- Répéter le processus pour le reste jusqu'à obtenir 0.

Exemple : $(45)_{10}$ en base 3.

- $3^3 = 27 \quad \Rightarrow \quad c_3 = 1, \quad 45 - 1 \cdot 27 = 18.$
- $3^2 = 9 \quad \Rightarrow \quad c_2 = 2, \quad 18 - 2 \cdot 9 = 0.$

Résultat : $(45)_{10} = (1200)_3$.

Remarque : Pour passer d'une base b à une autre base b', on peut utiliser la base 10 comme intermédiaire. Mais ce n'est pas forcément le plus pratique...

Valeur hexadécimal en bits

Principe : Chaque chiffre hexadécimal correspond à un groupe de 4 bits.

Décimal	Hexadécimal	Binaire
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Conversion facile de la base 16 vers la base 2

Principe : Il suffit de remplacer chaque chiffre hexadécimale par 4 bits.

- Exemple : $(1F3)_{16}$ en base 2.
  - $(1)_{16} = (0001)_2$
  - $(F)_{16} = (1111)_2$
  - $(3)_{16} = (0011)_2$
- Résultat : $(1F3)_{16} = (0001\ 1111\ 0011)_2$.
- Il ne reste qu'à supprimer les (éventuels) 0 inutiles à gauche de l'entier :

$$(1\ 1111\ 0011)_2.$$

Conversion facile de la base 2 vers la base 16

Principe : Grouper les bits par paquets de 4, en partant de la droite.

- Exemple : $(10111001101)_2 = (?)_{16}$
  - Grouper : 101 1100 1101.
  - Convertir chaque groupe :
    - $1101_2 = D_{16}$
    - $1100_2 = C_{16}$
    - $101_2 = 0101_2 = 5_{16}$
- Résultat : $(10111001101)_2 = (5CD)_{16}$.

Table des matières

1 Changement de base

2 Représentation finie

Représentation des entiers

- La notion mathématique d'entier n'est pas équivalente à la notion de type entier en informatique.
- En mathématiques :
  - Tout entier a un successeur qui est aussi un entier.
- En informatique :
  - Les entiers codés en machine ont une valeur maximale, déterminée par la taille (en bits) du type entier.
  - Au-delà de cette limite, un dépassement peut se produire (overflow).

L'addition n'est pas associative

Propriété mathématique :

- En mathématiques : $(a + b) - c = a + (b - c)$, pour tout $a, b, c \in \mathbb{N}$.

En informatique :

- $(a + b) - c \neq a + (b - c)$ si $a + b$ dépasse la capacité maximale.

Exemple : Entiers sur 8 bits

- $a = 200$, $b = 100$, $c = 50$.
- $a + b = 300$ (overflow, dépassement des 8 bits) $\rightarrow$ résultat incorrect car 300 ne peut pas être représenté.
- $a + (b - c) = 200 + (100 - 50) = 200 + 50 = 250 \rightarrow$ pas de problèmes...

La distributivité n'est pas toujours assurée

Propriété mathématique :

- En mathématiques : $a(b-c) = ab - ac$, pour tout $a, b, c \in \mathbb{N}$.

En informatique :

- $a(b-c) \neq ab - ac$ si ab dépasse la capacité maximale.

Exemple : Entiers sur 16 bits

- $a = 4000,\ b = 3000,\ c = 2999$.
- $ab = 12,000,000$ (overflow, dépassement des 16 bits) $\rightarrow$ résultat incorrect car $12,000,000$ ne peut pas être représenté.
- $a(b-c) = 4000(3000 - 2999) = 4000 \rightarrow$ pas de problèmes...

Entiers représentables

- Si on dispose de n bits pour coder un entier naturel, on peut représenter 2^n entiers.
- Les valeurs représentables sont comprises entre 0 et $2^n - 1$.

Exemples :

- $n = 16$: Entiers de 0 à 65 535.
- $n = 32$: Entiers de 0 à 4 294 967 295.
- $n = 64$: Entiers de 0 à environ $1,8 \times 10^{19}$.

Problème des nombres négatifs

- En informatique, les entiers signés doivent inclure des nombres négatifs.
- Quatre solutions courantes :
  - Signe Grandeur
  - (Complément à 1)
  - Complément à 2
  - Excentrement

Signe Grandeur (sur 8 bits)

- 1 bit pour le signe, les autres pour la valeur.
- Exemple :
  - $+24$ est codé 00011000
  - -24 est codé 10011000
- Problèmes :
  - Ne gère pas bien l'addition.
  - Deux codages possibles pour zéro ($+0 = 00000000$, $-0 = 10000000$).
  - L'intervalle des valeurs est donc $[-2^{n-1}+1, 2^{n-1}-1]$.

Complément à 1 (sur 8 bits)

- Pour les entiers négatifs, on prend le complément bit à bit.
- Exemple :
  - $+24$ est codé 00011000
  - -24 est codé 11100111
- Problèmes :
  - Ne gère toujours pas bien l'addition :
  - Exemple : $24 + (-1) = 00011000 + 11111110 = (0)00010110$.
  - Deux codages possibles pour zéro ($+0 = 00000000$, $-0 = 11111111$).
  - L'intervalle des valeurs est donc $[-2^{n-1} + 1, 2^{n-1} - 1]$.

Complément à 2 (sur 8 bits)

- Pour les entiers négatifs, on prend le complément à 1 bit à bit et on ajoute 1.
- Exemple :
  - $+24$ est codé 00011000
  - -24 est codé 11101000
- Avantages :
  - Gère correctement l'addition :
    $24 + (-1) = 00011000 + 11111111 = (0)00010111$.
  - Un seul codage pour zéro : $0 = 00000000$.
  - L'intervalle des valeurs est donc $[-2^{n-1}, 2^{n-1} - 1]$: on gagne une valeur négative car il n'y a qu'un seul zéro.

Remarque sur le complément à 2

- On parle de "complément à 2", mais il s'agit en réalité du complément à 2^n, où n est le nombre de bits.
- Pour représenter un entier négatif $-m$ sur n bits :
  - On code $2^n - m$, qui se situe entre 2^{n-1} et $2^n - 1$.
  - Le premier bit est toujours 1.

Remarque sur le complément à 2

- On parle de "complément à 2", mais il s'agit en réalité du complément à 2^n, où n est le nombre de bits.
- Pour représenter un entier négatif $-m$ sur n bits :
  - On code $2^n - m$, qui se situe entre 2^{n-1} et $2^n - 1$.
  - Le premier bit est toujours 1.

Par exemple, pour $m = 24$ et $n = 8$, $2^n - m = 256 - 24 = 232$.

Remarque sur le complément à 2

- On parle de "complément à 2", mais il s'agit en réalité du complément à 2^n, où n est le nombre de bits.
- Pour représenter un entier négatif $-m$ sur n bits :
  - On code $2^n - m$, qui se situe entre 2^{n-1} et $2^n - 1$.
  - Le premier bit est toujours 1.

Par exemple, pour $m = 24$ et $n = 8$, $2^n - m = 256 - 24 = 232$.

- $232 \div 2 = 116$ reste 0
- $116 \div 2 = 58$ reste 0
- $58 \div 2 = 29$ reste 0
- $29 \div 2 = 14$ reste 1
- $14 \div 2 = 7$ reste 0
- $7 \div 2 = 3$ reste 1
- $3 \div 2 = 1$ reste 1
- $1 \div 2 = 0$ reste 1

Remarque sur le complément à 2

- On parle de "complément à 2", mais il s'agit en réalité du complément à 2^n, où n est le nombre de bits.
- Pour représenter un entier négatif $-m$ sur n bits :
  - On code $2^n - m$, qui se situe entre 2^{n-1} et $2^n - 1$.
  - Le premier bit est toujours 1.

Par exemple, pour $m = 24$ et $n = 8$, $2^n - m = 256 - 24 = 232$. Donc $(232)_{10} = (11101000)_2$. De l'autre côté, $24 = 16 + 8 = 2^4 + 2^3 = (00011000)_2 \xrightarrow{\text{flip}} (11100111)_2 \xrightarrow{+1} (11101000)_2$. Le même résultat.

Types entiers en Java (Complément à 2)

Type	Taille (bits)	Plage des valeurs
`short`	16	$-32\,768$ à $32\,767$
`int`	32	-2.1×10^9 à 2.1×10^9
`long`	64	-9.2×10^{18} à 9.2×10^{18}

Excentrement

Si on travaille sur $k+1$ bits :

- On passe de l'intervalle $[0, 2^{k+1}-1]$ à l'intervalle $[-2^k+1, 2^k]$ par une **translation** de -2^k+1.
- Cela permet de coder les entiers négatifs en les décalant pour qu'ils soient dans une plage symétrique autour de zéro.
- **Exemple :** Si $k=3$, l'intervalle devient $[-4, 3]$ après translation.
- On verra plus de détails dans les prochains CM...